Building Expression Tree Algorithm Using Stacks

Given an infix expression, the algorithm builds an expression tree.

S is the input string (infix expression)
Opstack is operator stack
Tstack is tree stack

Steps:

If **S** empty, return empty tree
Else scan **S** from Left to Right and process each character (**Ch**) as follows:

1. If current **Ch** is operand
   Then add it to a node and push the node onto **Tstack.**

2. If current **Ch** = '('
   Then add it to a node and push the node onto **Opstack.**

3. If current **Ch** is operator
   Then
   3.1 If **Opstack** is empty OR TOP(**Opstack**) = '('
   Then add operator to node and push the node onto **Opstack.**
   3.2 If precedence of current operator **>** precedence of **Opstack** Top,
   Then add operator to node and push the node onto **Opstack.**
   Else Process top element of **Opstack.**
   3.3 Repeat step 3.2 until current operator node is pushed onto **Opstack.**

4. If current **Ch** = ')'
   Then
   4.1 While TOP(**Opstack**) != '('
   Then Process top element of **Opstack.**
   4.2 Pop matching ')' of **Opstack** and ignore it.

5. If end of **S** AND **Opstack** is NOT empty
   Then while !empty(**Opstack**) Process top element of **Opstack**.

Process top element of **Opstack:** (consider this a separate function)

1. Get top element of **Opstack** and connect it as the root for the top 2 elements in the **Tstack**. Note that the top element is right subtree and the second top element is the left subtree.
2. Pop the top 2 elements of the **Tpstack**.
3. Push the new subtree back onto **Tstack**.
4. Pop top element of **Opstack**.